Report

Roles -

Admin - This person can see and control everything every mess, every report, all the money stuff. They can add new staff and decide what each person is allowed to do. If something looks wrong anywhere, the Admin is the one who checks the record to find out who did what.

Store Keeper - This person manages the central store where all the ration is kept. When a vendor delivers rice, oil, dal, etc., the Store Keeper records it coming in. When a mess needs supplies, the Store Keeper sends it out and records that too. Their whole job is making sure the stock numbers are always correct. They don't get to see money/budget stuff that's not their job.

Mess Manager - This person runs just one mess. Every day, they type in how much ration was used and how many people ate that day. They can only see their own mess's information, not any other mess's. And they can never see the vendor payment screen that's simply not something they need to touch.

Finance Staff - This person tracks the budget, how much money was allotted, how much has been spent, and pays the vendors. They also pull together reports for whenever there's an audit. They can't edit stock numbers though that stays with the Store Keeper.

Vendor - This is the outside supplier. Later on, they'll get their own simple login where they can see their own orders and mark when a delivery is on the way. Nothing else. This part isn't being built yet it comes after everything else is working.

Techstack Used -

Frontend — React

- builds the actual screens people click on — Admin dashboard, Store Keeper's stock entry form, Mess Manager's daily consumption form, Finance's reports view. Also handles role-based routing, so a Mess Manager's app literally never loads the vendor-payment screen's code — not just hides it with CSS.

Backend — Node.js + Express (or Django REST Framework)

- the logic layer. Every "can this user do this action" check happens here. Handles things like: validating a PO receipt, calculating derived stock (received − issued − adjustments), running the forecast calculation, and writing to the audit log on every mutation. Django is worth considering specifically because its built-in permission system maps closely to your five roles.

Database — PostgreSQL

- the system of record for everything — vendors, purchase orders, stock transactions, consumption records, budgets, audit log. Chosen because it gives you real ACID transactions (so "issue 50kg rice" can't half-complete and leave phantom stock) and Row-Level Security (so a Mess Manager literally cannot query another mess's data, enforced by the database itself, not just your API code).

Redis

- caching only — speeds up things like the Admin dashboard's aggregate numbers (total stock across all messes, budget totals) so they don't get recalculated from scratch on every page load. Also good for storing session/token data if you want fast login checks. Never holds the actual source-of-truth data.

JWT (JSON Web Tokens)

- authentication — proves who's logged in and what role/mess they belong to, without the server having to look up a session on every single request. Short-lived access token + refresh token, so a stolen token expires quickly.

TimescaleDB

- not needed on day one — it's a PostgreSQL extension you'd bolt on later if `StockTransaction` and `ConsumptionRecord` grow into millions of rows. It auto-partitions by time so your reporting queries stay fast without you manually managing partitions.